



ARCHITECTURE & DESIGN SPECIFICATION

WeVibe Network

Verified Memory for
Coding Agents — **Owned**
by No One, **Killable by No**
One.

A censorship-resistant knowledge network where hard-won engineering fixes are encrypted on-chain, curated by accountable human experts, and delivered straight into any coding agent — readable by no one in the middle, deletable by no one at all.

CONTENTS

00 Abstract

01 Overview

- 1.1 What WeVibe Is
- 1.2 How It Works at a Glance
- 1.3 Who It Is For

02 Design Philosophy

- 2.1 The Problem
- 2.2 The Six Questions of Shared-Memory Trust
- 2.3 The Inversion
- 2.4 The Organization Model
- 2.5 Human-in-the-Loop
- 2.6 Protocol, Not Platform
- 2.7 Verification by Pre-Commitment

03 System Architecture

- 3.1 Entities and Roles
- 3.2 The Three Software Pieces
- 3.3 Organization Lifecycle
- 3.4 Tool Surface

04 Cryptographic Architecture

- 4.1 Threat Model
- 4.2 Epoch-Based Key Hierarchy
- 4.3 Memory Encryption and Moderation Keys
- 4.4 Key Distribution: Sealed Envelopes
- 4.5 Hub Confidentiality
- 4.6 Metadata Visibility Model
- 4.7 The Memory Sanitization Pipeline

05 Retrieval and Recall

- 5.1 The Retrieval Pipeline
- 5.2 Recall Flow

06 Local Architecture

- 6.1 Local Footprint
- 6.4 Contribution Flow
- 6.6 Plugin Architecture
- 6.7 Chain-Resolved Hub Endpoints

07 Content Review and Accountability

- 7.4 The Contributor-Signed Canonical Body
- 7.5 The Report Model
- 7.7 No Internal Courtroom

08 Provenance, Reputation, and the Social Graph

- 8.3 Identity and Attribution Model
- 8.8 Public Discovery Interface
- 8.11 Session Provenance: The Honest Boundary
- 8.12 Goal-Sealed Trajectory Verification

09 Storage Architecture

10 Security Analysis

- 10.7 Organization Capture and Public Escalation
- 10.8 Receipt Integrity and Predicate Gaming

11 Decentralized Architecture

12 References

A Cryptographic Procedures

B Badge & Reputation Taxonomy

C The Document Set

00 Abstract

WeVibe is a memory layer for AI coding agents that makes shared memory usable across trust boundaries. It delivers version-exact fixes and negative knowledge as attributed memories: each item is tied to its author, its organization, and a curation decision.

Memories are extracted from real sessions and encrypted before submission; contribution requires explicit user actions. Approved memories are committed as encrypted chain state that validators replicate without plaintext access. At recall, candidates are decrypted locally, scanned by the local sanitization pipeline, and presented through the plugin-controlled injection gate – every recalled memory passes human eyes before it enters an agent's context.

Curation is accountable without a platform tribunal: each organization's leader is the sole chain publisher for that organization's commits. Consumers can file wallet-signed on-chain reports that a captured organization cannot suppress; dismissal or inaction unlocks a reporter-signed public plaintext reveal verified against a contributor-signed anchor.

Confidentiality is stated narrowly and precisely: the hub that coordinates retrieval cannot decrypt stored memory ciphertext under the Umbral proxy re-encryption relation. This is not a claim that the hub learns nothing; for retrieval the hub holds embeddings and keyword metadata that constitute a disclosed semantic shadow, with operational mitigations rather than a zero-knowledge index.

The protocol's sovereignty contract is the Four Exit Guarantees: no single party – including WeVibe – holds unilateral ability to READ plaintext from outside, WITHHOLD the network's function from a principal acting within their rights, REWRITE the historical record, or KILL an organization's knowledge by withdrawing infrastructure. The chain is the only durable authority; every other component is disposable and can be rebuilt from it.

*Nothing enters an agent's context without **human eyes on it first**.*

This document is the architecture contract: the normative specification the network is built and audited against.

01 Overview

1.1 What WeVibe Is

WeVibe is a memory layer for AI coding agents. A local plugin and MCP path retrieve relevant community memory, present each candidate to the user, and inject only approved memory. A memory is one atomic insight: what to do, when it applies, and what not to do.

Memories are extracted from real sessions, encrypted before submission, reviewed by accountable humans, and committed as attributed encrypted chain state.

1.2 How It Works at a Glance

FIGURE 1 • THE WEVIBE LOOP

1 • Contribute – A developer extracts a memory from a real session and encrypts it on their own machine.



2 • Curate – An organization's expert leader reviews it and signs it onto the chain.



3 • Recall – Another developer's agent surfaces it, they approve it, and it enters the session.

☞ *Every use is attributed back to the author – the loop closes.*

🔒 Encrypted, on-chain, owned by no one.

Figure 1. Contribution, curation, and recall form one loop; attribution closes it.

1.3 Who It Is For

ROLE	WHAT THEY DO	WHAT THEY NEED
Developer	Codes with an agent, recalls community memory, and chooses what to contribute.	A passkey (Face ID / fingerprint). No wallet, no seed phrase.
Organization leader	A domain expert who curates memory quality and signs every commit.	Holds the organization's scarce slot.
Validator	Runs consensus and stores all chain state, including encrypted memories.	Runs a node.
WeVibe (the protocol)	Open-source software. No company sits in the middle; the protocol depends on no single entity.	Nothing — it is code.

Table 1. The four roles. The developer is the protagonist; the organization is the container.

02 Design Philosophy

2.1 The Problem

Shared agent memory becomes valuable only when it crosses trust boundaries. A memory you did not personally witness is an untrusted write from a stranger. It is usable only if six questions can be answered: **authenticity, correctness, safety, confidentiality, permanence, and sovereignty**.

Developers solve hard problems with agents every day, yet sessions repeatedly reset. Version-exact fixes and negative knowledge are rediscovered, repaid for, and lost in private transcripts.

This is most visible on local or smaller models: they generate code, but they do not carry a stack's lived edge cases by default. The scarce input is verified memory with provenance, not more generic text.

Developer channels are now saturated with plausible AI-generated content that misses decisive details. Provenance – who wrote it, who curated it, and under what accountability path – becomes the scarce input.

Prior shared-knowledge systems lived in company databases with kill switches. Knowledge that should outlive its platform needs infrastructure that cannot be unilaterally withdrawn.

Cross-run agreement is expensive and repeatedly repurchased; verified shared memory amortizes that cost across the network.

2.2 The Six Questions of Shared-Memory Trust

The rest of this specification is structured as explicit answers to the six trust questions:

01 Authenticity – is this really from who it claims? → contributor signatures + wallet identity; the contributor-signed anchor (§7.4) and the provenance/two-key model (§8). The remaining gap – proving the *session* itself – is stated openly in §8.11.

02 Correctness – is it right? → accountable human curation by the organization leader, Earned-Trust decay, and goal-sealed trajectory verification where the work admits an executable check (§2.5, §2.7, §7, §8.12).

03 Safety – will it harm my agent? → local wevibe-guard sanitization plus a mandatory human injection gate (§4.7, §2.5 / §5.2).

04 Confidentiality – who can read it? → Umbral proxy re-encryption; the hub cannot decrypt stored ciphertext, with the semantic-shadow boundary stated openly (§4.5).

05 Permanence – will it survive? → approved memories are encrypted chain state replicated by validators (§3.2 / §9.1).

06 **Sovereignty** – can any single party read, withhold, rewrite, or kill it? → the Four Exit Guarantees; the chain is the durable authority and every other component is disposable (§2.3, §11).

Landscape: memory vs. shared memory you can trust. Most shipping "memory" systems are personal or single-organization stores; useful inside one trust domain, but they do not address the requirements that appear when memory is shared across strangers – especially provenance/authenticity and a human gate before injection. The scorecard below is factual, drawn from public product documentation.

LEGEND ✕ Not present – Partial ✓ Present

SYSTEM	AUTH	CORRECT	SAFETY	CONFID	PERM	SOV	HUMAN-GATE
ChatGPT Memory (OpenAI) [16]	X	—	X	X	X	X	Auto-inject (no gate)
Claude Memory + Projects (Anthropic) [17]	X	—	X	X	X	X	Auto-inject (no gate)
Gemini Personal Intelligence (Google) [18]	X	—	X	X	X	X	Auto-inject (no gate)
GitHub Copilot Memory [19]	X	—	—	X	X	X	Auto-inject (no gate)
Cursor / Windsurf / Cline / Roo [20]	X	X	X	—	—	—	User-edits-own-only
MemO [21]	X	—	X	X	—	X	Auto-inject; dev-configurable
Letta / MemGPT [22]	X	X	X	X	—	X	Auto-inject (no gate)
Zep / Graphiti [23]	X	—	X	X	—	X	Auto-inject (no gate)
LangMem / LangGraph [24]	X	—	X	X	—	X	Auto-inject / retrieval; dev-config
RAG-over-wiki (Notion/Confluence) [25]	X	X	X	X	X	X	Retrieval-only; dev-config
Recall Network [26]	✓	—	X	X	✓	—	Auto-inject (no human gate)
Ceramic / ComposeDB [27]	✓	X	X	X	✓	—	Not addressed (data layer)
Arweave + AO [28]	X	X	X	X	✓	—	Not addressed (storage layer)
GAIA / GaiaNet [29]	—	X	X	—	—	—	Not addressed (single-node)

Table 19. Landscape as of July 2026; scored from public product documentation (see §12). Axis values are reported facts, not editorial ranking.

The structural result is stable across categories: established systems are single-trust-domain. Either one vendor holds readable plaintext, or "self-hosting" relocates the single trusted reader to one operator — so "sharing" is relocated centralization, not decentralization. Team/workspace sharing is not admission of a stranger's contribution with independent verification. Decentralized efforts have advanced authenticity and permanence, but not the safety problem of stranger-contributed plausible-but-wrong or malicious memory. And every system above with an injection step auto-

injects: none places a human between a stranger's contributed memory and another party's agent context – the No-Blind-Injection requirement this paper treats as non-negotiable.

2.3 The Inversion

Two decades of file-sharing networks proved a property that no legal or infrastructural pressure has reversed: when content is replicated across many independent machines and the index is owned by no one, distribution cannot be stopped. That is exactly the property durable knowledge sharing needs – and exactly what no knowledge platform has ever had.

But that lineage carried two fatal absences. It moved content *without the consent of its creators*, and it *stripped provenance* – the one thing that makes technical knowledge trustworthy. WeVibe is the inversion: keep the unstopability, and restore consent and attribution.

- **Consent is structural.** Nothing leaves a contributor's machine without two explicit actions – Extract, then Submit (§6.4). Sessions stay local by default.
- **Provenance is the payload.** Every memory carries contributor signature, organization curation, and serve history.
- **Unstopability is enforced as four exit guarantees**, defined below.

DEFINITION The Four Exit Guarantees

No single party – including WeVibe-the-company, any hub operator, or any organization leader – may hold the unilateral ability to:

01 • READ

a member's memory plaintext from the outside;

02 • WITHHOLD

the network's function from a principal acting within their rights;

03 • REWRITE

the historical record; or

04 • KILL

an organization's knowledge or a contributor's standing by withdrawing infrastructure.

The chain is the only durable authority; every other component must be disposable and reconstructible from chain state plus member-held keys.

2.4 The Organization Model

Organizations are domain-expert-run memory collections. Leaders define standards, manage membership, and publish with sole chain signature authority.

ATTRIBUTE	DESCRIPTION
Membership roster	Managed by the leader.
Role hierarchy	Leader, Reviewer, Member.
Commitment standards	What counts as high-quality memory in this organization.
Domain focus	The organization's subject and coverage map.
Operating policies	Contribution/review cadence and recall access.

Table 2. An organization is a collaboration container with its own standards.

Public plaintext keywords are discovery labels (`redis` , `solana` , `django`) and are intentionally non-secret metadata (§4.6).

KEY · PERSONAL MEMORY VS SHARED ORGANIZATION MEMORY

Shared organization memory is the curated, encrypted, chain-anchored corpus. Personal memory is a bounded local pull layer and sits outside the chain-rebuildable contract.

2.5 Human-in-the-Loop: The Curator Workbench and the Plugin Gate

WeVibe is a curator workbench, not an autonomous ranking machine. Curators process retrieval frequency, denials, staleness, and drift signals; they decide what enters and stays in the corpus.

On the consumer side, one invariant governs everything: **every memory passes through human eyes before it enters an agent's context.**

During a coding session the plugin harvests local signals, auto-queries organization memory through the hub, decrypts candidates locally, scans them with wevibe-guard, and presents an approval gate:

FIGURE 2 · THE MEMORY INJECTION REQUEST

● ● ●

Memory Injection Request

"Redis cluster-node-timeout must be set to 15000ms when running behind AWS NLB with cross-AZ failover..."

Contributor	wevibe1x7k...f3q2	Wallet age	8 months
Rep score	347 (Tier 3)	Serves	214 across 12 orgs
Domain	redis, kubernetes, aws	Verification	T2 test-backed
DETECTIONS	[url: aws.amazon.com]		

✓ Accept

✕ Deny

⊘ Block

🚩 Report

Figure 2. The plugin approval gate. The consumer sees the memory, the security flags, and who wrote it before deciding.

BUTTON	EFFECT	SIGNAL EMITTED
Accept	Memory is injected into agent context. Serve attribution is queued to chain aggregates for the contributor and organization.	Public serve attribution. No per-serve payout.
Deny	Memory is blocked for this session/context only.	None. A neutral "not what I need right now" signal — not a corpus down-vote; no chain event.
Block	Memory enters the consumer's permanent personal blacklist.	A global corpus denial signal that feeds Earned-Trust decay (\$7.7). This is the load-bearing negative path.
Report	Memory is reported on-chain against the contributor, with the reporter's wallet, and enters the organization's accountability path (\$7.5–\$7.7). The memory keeps serving until the report resolves.	A wallet-gated, gas-paid on-chain accountability event.

Table 3. Reports are the high-friction accountability primitive; Block is the low-friction ranking signal. The Deny/Block split is load-bearing: Deny says nothing about quality, Block drives decay.

Two properties make the gate trustworthy. **No plugin installed means no memory injection path** – the MCP server has no route to force memory into an agent's context without the plugin front end. And injection is **per session, not per turn**: recall is queried on each user prompt, but a recalled

memory is injected once per coding session (keyed to the session identifier), not re-pushed on every model turn; a relevance floor and surface budget keep the injected volume small.

2.6 Protocol, Not Platform

WeVibe is a protocol with open, auditable surfaces. The chain, hub, local client, and plugin each perform one narrow role.

#	CAPABILITY	DESCRIPTION
1	On-chain encrypted storage + provenance	Memories are committed as encrypted blobs with attribution metadata.
2	Human-gated delivery	The plugin is the mandatory approval path before any memory enters agent context.
3	Public attribution	Contribution and serve aggregates power the provenance and reputation surfaces.
4	Domain-expert governance	Leaders and reviewers curate memory quality inside each organization.
5	Suppression-proof accountability	Consumer reports and public escalation broadcast directly to the chain; no WeVibe infrastructure sits between a reporter and the record (\$7.5).
6	Coordination layer	wevibe-hub runs hosted coordination, accounting, and retrieval; it is never a plaintext memory oracle.
7	Local retrieval edge + sanitization	Decryption, guardrails, and injection run close to the user.
8	Context injection format	Approved memories are packaged for direct agent-context use.

Table 4. Eight protocol capabilities, split across four cooperating components.

2.7 Verification by Pre-Commitment: The Verification Goal

Human review and reputation are judgment. Judgment scales poorly across trust boundaries, and machine judgment fails in a specific, measured way: in judge-gated workflow-memory systems [35], roughly half of certified inductions have been found to originate from failed trajectories [34]. In a trustless network the failure is structural, not statistical – the judge runs on the contributor's machine, so a verdict from the contributor's own model is self-attestation with extra steps. WeVibe therefore assigns the extraction LLM a narrow role: it drafts and annotates candidate memories; it never gates them.

Where the work admits an executable definition of "done," WeVibe replaces the judge with a fact, established in three moves:

01 Seal the target before the work. At session start the plugin seals the goal, an executable test predicate that defines completion, and a hash of the starting working tree – signed and timestamped before the outcome exists.

02 Record the trajectory as a hash chain. Working-tree states are hashed and chained across sessions until the sealed predicate passes.

03 Let the predicate speak. The sealed test's exit status – not a model's opinion, not the contributor's claim – determines whether the goal was met.

A target fixed before the answer existed cannot be retrofitted to whatever was produced. And fabrication buys nothing: to forge a passing receipt, a contributor must make the pre-sealed predicate actually pass – which is the work itself. Verification cost collapses onto honest cost.

In a trustless setting, the only thing a stranger can verify later is something you locked in before you knew the answer.

KEY The Verification Goal

For every memory whose claim can be checked by execution, WeVibe's goal is to carry a receipt a stranger can check: sealed before the outcome existed, graded openly by tier, and used only as a label – never as a contribution gate. Facts replace judgment wherever a fact can exist; judgment – leader review, the human gate, reputation – covers everything facts cannot reach.

The mechanism, its receipts, its multi-session semantics, and its honest boundaries are specified in §8.12.

03 System Architecture

3.1 Entities and Roles

Let **L** be leaders, **O** organizations, **D** reviewers, **C** contributing members, and **R** read-only members. Each organization $o \in O$ has a leader $l(o) \in L$ who controls membership and configuration.

Participants hold Ed25519 identity keys plus X25519 encryption keys. Contributor public keys are on-chain; serve receipts and reputation aggregates are keyed to those identities.

ROLE	PERMISSIONS	APPOINTED BY
Leader	Full organization control: roster management, epoch rotation, key custody, reviewer appointment, keyword-taxonomy management, slot/rent/revenue management.	Self (slot acquisition).
Reviewer	Review pending memories; approve/deny; decrypt pending submissions via SK_mod(e).	Leader.
Member	Submit memories (within per-epoch bandwidth caps); retrieve approved content; view own pending submissions.	Leader (via invitation).

Table 5. All roles require epoch-specific encryption keys for content access; the leader distributes these through sealed envelope key exchange (Appendix A).

3.2 The Three Software Pieces

DIAGRAM 1 · SYSTEM COMPONENTS AND TRUST

wevibe-chain Source of truth.

Sees ciphertext, never plaintext.

‡ derived from chain state

wevibe-hub Coordination + retrieval.

Disposable. Never a plaintext oracle.

‡ encrypted candidates + query vectors

LOCAL MACHINE

MCP server + plugin Safety + approval + injection.

The only place plaintext lives at recall.

plugin $\neq$ chain: serve receipts / RPC

Diagram 1. Three software pieces over one chain. Plaintext is confined to the local machine.

3.3 Organization Lifecycle

Creation. The leader acquires a governance-capped slot (hard cap: 32 alpha / 320 testnet / 3200 mainnet), signs `MsgRegisterOrg` from their own wallet, creates `K_master`, and derives epoch-0 keys and moderation keys.

Operation. Members join by leader approval. The leader distributes sealed key envelopes for the current epoch.

Rotation. Removing a member sets `rotation_pending` and requires epoch advancement.

- 1 **Removal triggers `rotation_pending`.** The chain marks the organization pending rotation; the removed member's envelope is deleted.
- 2 **New submissions are buffered.** Contributors can still submit, but submissions enter a hub-side rotation buffer – not admitted to the chain, not indexed for retrieval, not assigned a final epoch.
- 3 **Leader completes rotation.** The leader derives new epoch keys from `K_master` via HKDF, generates a new moderation keypair `SK_mod(e+1)/PK_mod(e+1)`, and re-seals envelopes for all remaining members.
- 4 **Buffer finalizes.** After rotation completes, buffered submissions are released to the chain under the new epoch.
- 5 **Grace-period escalation.** If rotation is not completed within 72 hours, the organization's submission bandwidth is suspended until it is.

Table 6. Removing a member advances the epoch. Rotation provides forward secrecy only — a removed member retains previously distributed epoch keys and can still decrypt content from their membership period.

KEY • WALLET-FREE CONTRIBUTOR ONBOARDING

A contributor can create an account with a passkey (Face ID / fingerprint), install the plugin, and join an organization without a wallet. Contribution remains explicit and dashboard-driven with two consent gates: **Extract** then **Submit**. A wallet is optional later for rewards and mainnet fees; the leader still requires a wallet to register and hold the organization slot.

Chain state is authoritative for membership transitions; the hub mirrors chain events.

3.4 Tool Surface

The plugin registers operational tools in the coding agent. Recall is automatic in the plugin path (no agent recall tool call); contribution submission stays in dashboard flows.

TOOL	PURPOSE
<code>setup_org</code>	First-run organization bootstrap when the local stack has no membership. Guides the create/join flow and local setup handoff.
<code>wevibe_status</code>	Show organization membership and runtime status so the user can verify local setup and connectivity.
Consumer-settings tools	Configure the content filter: <code>[Implementations + DNDs]</code> or <code>[DNDs only]</code> . Default: <code>[Implementations + DNDs]</code> .

Table 7. Agent-visible tools. The MCP server backend handles recall mechanics — query construction, local embedding/decryption, guard scanning, and packaging candidates for the plugin gate — and is not directly callable by the agent. It never auto-submits contributions.

04 Cryptographic Architecture

4.1 Threat Model

ADVERSARY	WHAT THEY CAN SEE	WHAT THEY MUST NOT DO
Chain validators	Ciphertext, organization IDs, contributor public keys, serve metadata.	Read memory content.
Unauthorized external observers	Public chain state (no epoch keys).	Decrypt content.
Removed members	Content from their membership period (they held epoch keys).	Decrypt content created after removal (forward secrecy).
Memory poisoning via recalled content	— (a malicious member submits an indirect prompt injection that passes review).	Reach an agent's context unseen. The human gate is the final defense: the reviewer and consumer see every memory, plus the contributor's reputation and wallet age.
Receipt forger / predicate gamer	Their own seals, chains, and receipts (produced on hardware they control).	Pass a fabricated or hollow receipt off as strong verification. Tier semantics cap the claim (§8.12); a forged receipt against a sealed predicate becomes a provable, reportable lie on reveal-and-replay.

Table 8. The five adversary classes the system defends against.

⚠ WARNING · OUT OF SCOPE (EXPLICITLY NOT DEFENDED)

The system does **not** protect against: a compromised active member who leaks epoch keys or decrypted content; metadata inference from on-chain patterns (organization sizes, submission frequency, serve patterns); compromised reviewer endpoints; or semantic payloads encoded in natural-language prose. It also does not prevent fabrication of T1–T2 verification receipts on contributor hardware, nor trivially weak sealed predicates; both are bounded by label semantics, curation, and dispute-path replay rather than cryptography (§8.12).

4.2 Epoch-Based Key Hierarchy

Per-epoch keys derive from organization `K_master` via HKDF. The leader derives keys locally and never transmits epoch secret keys to the hub. Full derivation formulas and invariants are in Appendix A (CODE 1).

4.3 Memory Encryption and Moderation Keys

Each memory uses a random per-memory DEK. The DEK is initially sealed to the moderation public key for review, then re-wrapped to the epoch encryption key on approval. Full procedures are in Appendix A (CODE 2 and CODE 3).

4.4 Key Distribution: Sealed Envelopes

Epoch keys are distributed to members via sealed envelopes (`seal_to_pubkey`), with custody and recovery constraints anchored at the organization level. Full envelope procedure and recovery model are in Appendix A (CODE 4).

4.5 Hub Confidentiality: Why the Hub Cannot Decrypt

A natural objection: if an authorized consumer can retrieve plaintext, surely a captured hub could too. This section states the guarantee narrowly and then states the boundary.

The claim, stated narrowly. The hub **cannot decrypt** memory content, including under a fully malicious-hub model. This is not a claim that the hub learns nothing.

*The hub is a postal sorting office that re-addresses a sealed envelope so a different recipient's key opens it – **without ever being able to open it itself.***

The confidentiality core (Umbral proxy re-encryption over secp256k1). Umbral layers verifiability and threshold-splitting on top; the core is the ElGamal-style re-encryption relation below. Let `G` be the curve generator and `n` the curve order.

CODE 5 · THE RE-ENCRYPTION RELATION

Keys

org (delegator)	secret $a = \text{epoch_sk}$	public $A = a \cdot G = \text{umbral_pk}$
member (recipient)	secret $b = \text{receiving_sk}$	public $B = b \cdot G = \text{pre_pubkey}$

Encrypt (client-side, under the org public key A)

the per-memory DEK is sealed in a capsule; the capsule public part is a point $(E+V)$

sealed key $K = \text{KDF}(a \cdot (E+V))$ ← recovering K the direct way needs a

Re-encryption key (kfrag – minted ONLY by the leader, REQUIRES the secret a)

$rk \approx a \cdot b^{-1} \pmod{n}$ ← reveals neither a nor b (discrete log);

cannot decrypt by itself

Re-encrypt (the hub's ONLY crypto op: capsule + kfrag, NO secret key)

$cfrag = rk \cdot (E+V) = (a \cdot b^{-1}) \cdot (E+V)$

Decrypt (client-side, on the member's device, REQUIRES the secret b)

$b \cdot cfrag = b \cdot (a \cdot b^{-1}) \cdot (E+V) = a \cdot (E+V) \rightarrow K = \text{KDF}(a \cdot (E+V))$
then AES-decrypt the memory body with the recovered DEK

- **Encryption** seals the per-memory DEK under the organization public key A ; recovering the sealed key $K = \text{KDF}(a \cdot (E+V))$ the direct way requires the organization secret $a = \text{epoch_sk}$, which the hub never holds.
- **The kfrag** encodes $rk \approx a \cdot b^{-1}$ and can be minted only by a party holding the delegating secret a – the leader. Recovering a or b from rk is the discrete-logarithm problem, and rk decrypts nothing on its own.
- **Re-encryption** is the hub's *only* cryptographic operation: given the capsule and a leader-minted kfrag, and no secret key, it computes $cfrag = rk \cdot (E+V)$.
- **Decryption** happens only on the member's device, which multiplies the cfrag by its own secret scalar b to recover K , hence the DEK.

KEY • THE PUNCHLINE

The hub can compute $a \cdot b^{-1} \cdot (E+V)$ but needs $a \cdot (E+V)$. The one missing operation is a single multiplication by b , the member's secret scalar, which the hub is *structurally* never given. This is geometry, not policy: no configuration, key rotation, or privileged mode grants the hub b . Everything the hub holds – {capsule, cfrag, ciphertext, kfrag, A , B } – is computationally independent of the plaintext without b . This is exactly Umbral's IND-PRE-CCA security guarantee [15].

Attacks a malicious hub might attempt.

- *"The hub forges its own kfrag toward a key it controls."* **Fails**. Minting a kfrag requires the delegating secret $a = \text{epoch_sk}$, which the hub never holds. The hub can only *apply* leader-minted kfrags, and those are minted toward the registered public keys of authorized members – never toward a hub-controlled key.
- *"The hub colludes with an authorized member."* **Conceded openly**. An authorized member decrypts with their own secret b and could leak the plaintext. That is authorized-insider abuse, not a cryptographic break, and it is already out of scope in the threat model (§4.1). The hub as a standalone party still cannot read.

⚠ WARNING • THE HONEST BOUNDARY (WHAT THIS DOES NOT COVER)

Two statements must not be conflated: **TRUE (proven above)**: the hub cannot **decrypt** your memory content. **FALSE (never claimed)**: the hub learns **nothing** about your content.

For search, the hub holds – in Qdrant – clean float32 embeddings plus plaintext keyword weights: a disclosed, lossy, realistically-invertible **semantic shadow** of each memory (§4.6, §9.3). Published embedding-inversion research [13][14] shows approximate content recovery from clean embeddings is realistic. The mitigations here are **operational, not cryptographic**: Qdrant API auth, internal-network deployment, per-organization collection isolation, and signed responses. Encrypted vector search is the documented evaluation trigger – formally evaluated when an organization requests confidentiality-sensitive hosting, or when public-testnet launch planning begins, whichever comes first.

WeVibe makes **no** claim of a zero-knowledge index or a content-confidential hub. "Cannot decrypt" and "learns nothing" are different guarantees, and only the first is made.

Two further notes. **The consumer-side injection gate is live and enforced today** – a blocking, fail-closed human-approval step (Accept/Deny/Block/Report, §2.5) that injects only human-approved memories; benchmark/test mode outside the shared-memory safety contract may auto-approve for evaluation only. **Key locality:** the hub never receives `epoch_sk` ; only the epoch public key and finished kfrags cross the wire, and the confidentiality proof rests on this locality holding cleanly.

4.6 Metadata Visibility Model

WeVibe organizations are public developer communities. On-chain metadata is intentionally public for discovery and reputation.

ON-CHAIN (PUBLIC BY DESIGN)	LOCAL TO THE MCP/PLUGIN (HUB NEVER SEES)
Organization IDs and topic tags; contributor public keys; encrypted memory blobs; plaintext keyword terms and weights; submission timestamps; memory sizes; epoch boundaries; serve receipts (batched per epoch); reputation aggregates; bandwidth consumption; quarantine state; report flags.	Decrypted memory plaintext; local wevibe-guard and blacklist state; session context profiles.

Table 9. Embedding vectors and keyword-weight metadata live in the hub’s Qdrant (§9.3); the hub stores ciphertext and vectors but never decrypts.

4.7 Defense-in-Depth: The Memory Sanitization Pipeline

Sanitization is canonical here and referenced elsewhere. Decryption, scanning, approval, and injection all run locally.

DIAGRAM 2 · THE SANITIZATION PIPELINE

SUBMISSION TIME (before on-chain storage)

1 wevibe-guard scan. Rule-based detection (YARA-X + regex) for injection patterns, credential detection, and exfiltration matching. Advisory: it warns but does not block. The human reviewer is the security boundary.

2 Text sanitization suite. Normalizes and flags injection vectors invisible to the human eye: zero-width and unusual-whitespace characters, emoji-pattern "sleeper" payloads, homoglyph substitutions, bidirectional/directional-override sequences, invisible Unicode tag characters (U+E0000 block), and mathematical-alphanumeric injection (U+1D400–U+1D7FF).

3 Encryption. Memory encrypted with a per-memory DEK; the DEK sealed to the moderation public key.

4 Human review. The reviewer decrypts locally, reads plaintext, runs a steganography scan, and approves or denies.

5 On-chain submission. The approved memory (ciphertext + wrapped DEK + metadata) goes on-chain and is mirrored to hub storage for retrieval. The organization pays the submission cost.

↓ on-chain → recall

RECALL TIME (before delivery)

6 Hub candidate query. The MCP/plugin posts the local query vector to hub retrieval and receives memory IDs, metadata, and matched keywords.

7 Ciphertext fetch + local decryption. The MCP/plugin fetches each candidate's ciphertext and decrypts it locally through the Umbral sidecar.

8 Blacklist filter. The MCP/plugin checks the local blacklist and chain quarantine flags.

9 wevibe-guard scan. The same scan on the decrypted memory, catching payloads not detectable at approval time (new rules since approval).

10 Text sanitization suite. The same text normalization and invisible-character detection, applied to the decrypted memory.

11 Artifact extraction and egress flagging. Typed extraction of URLs, bare domains, IPv4 addresses, shell commands, package-install commands, and config directives. Every network-resolvable token is flagged for the reviewer (advisory – the human gate decides).

12 Plugin approval gate. The plugin renders the approval UI with guard detections and contributor trust signals. The user sees the memory, the flags, and who wrote it, and decides.

13 Serve receipt. Each serve/denial entry is signed by the consumer's per-organization serve key (Ed25519, offline – no wallet, no gas); the batch is carried by the organization serving key under feegrant. The chain verifies each entry's signature, recomputes the dedup fingerprint itself, and counts only verified entries (§8.3).

14 Context injection. Approved memories are formatted as `context:\n{memory content}` and injected into the agent prompt.

Diagram 2. Sanitization runs twice — once before storage, once before delivery — because rules improve over time.

CATCHES

Rule-based prompt injections (YARA-X + regex); credential leakage (AWS keys, API tokens, passwords, connection strings); invisible-to-the-eye Unicode steganography — zero-width and unusual-whitespace characters, homoglyph substitutions, bidirectional and directional-override sequences, invisible Unicode tag characters (U+E0000 block), and mathematical-alphanumeric injection (U+1D400–U+1D7FF); emoji-pattern sleeper-prompt payloads; Base64-encoded injections; external URL injection (scheme-ful and scheme-less); bare hostname references (any TLD); IPv4 literal references; malicious dependency injection; config-directive injection; shell pipe-to-execution attacks; previously-rejected memories (local blacklist + chain quarantine).

DOES NOT CATCH

Semantic payloads encoded in natural-language prose (mitigated by human review and contributor reputation). Technically-plausible but subtly wrong recommendations (mitigated by reviewer domain expertise and contributor reputation visibility).

Table 10. The pipeline hardens against mechanical attacks; human judgment covers the semantic residue.

05 Retrieval and Recall

Retrieval is hub-served; plaintext handling remains local. The hub retrieval plane (Qdrant index + ranking) is derived and rebuildable from chain state plus organization keys.

5.1 The Retrieval Pipeline

Context profiling. Session context (intent, stack, dependencies, errors, files) is harvested locally and used for pre-filtering.

Keyword extraction. Contributor extraction proposes weighted keywords; leaders curate taxonomy at batch stage. Keywords are retrieval bonus terms, never hard gates.

Retrieval representation (situation-centric card, symmetric embedding). Memory and query are embedded from deterministic cards with matched `atomic-embed-text` prefixes (`search_document:` and `search_query:`). Memory vectors are produced at approval on a plaintext-capable client path, not by the hub.

Semantic embedding. Query vectors are computed locally via Ollama and posted to hub retrieval.

DEFINITION Atomic memory format

Each memory is a single, self-contained technical insight with four fields:

- `implement` – the fix itself: specific technical knowledge in 1–2 sentences, with exact values.
- `context` – the environment, versions, and conditions where it applies.
- `dnd` – negative knowledge: what *not* to do, and why.
- `stack` – the specific technologies involved.

Retrieval scoring.

CODE 6 · SCORING

```
keyword_boost = Σ(query_weight_i × memory_weight_i)
final_score    = vector_score + min(γ × keyword_boost, δ × vector_score)

Defaults: γ = 0.1, δ = 0.15
```

Vector similarity is primary; keyword overlap is additive and capped. Parameters (`γ` , depth, `ε`) are tuning defaults, not protocol constants.

Blacklist filtering + fetch/decrypt. Local blacklist and quarantine flags apply before approval. Candidates are fetched as ciphertext and decrypted locally through Umbral.

Contested-result handling. When the top two results are near-tied (score gap < 0.20), the shipped default is **deterministic twin-suppression** – surface the clear winner and suppress the near-tied twin (no model call). A non-blocking, model-based rerank via a separately configured endpoint is an optional, deferred enhancement, not part of the shipped path.

5.2 Recall Flow (Auto-Query, Plugin-Gated Injection)

DIAGRAM 3 · RECALL FLOW

LOCAL PLUGIN

- 1 Developer works in their coding session.
- 2 Plugin harvests local session signals (intent, task, stack, dependencies, errors, files) and attaches the session to any open goal seal on this repo (§8.12).
- 3 Plugin auto-submits the recall query via local MCP (no agent recall tool call).
- 4 Build the deterministic need-card + keyword params → compute query embedding locally via Ollama (`nomic-embed-text`) → POST the query vector to the hub retrieval endpoint.



HUB

- 5 Qdrant vector + keyword-ranked search over hub-hosted embeddings → return encrypted candidates + ranking/trust metadata.



LOCAL PLUGIN

- 6 Decrypt candidate ciphertext via the Umbral sidecar → run wevibe-guard + policy checks → apply content filter → render candidate details (memory text, score, matched keywords · guard result · contributor stats · memory stats · report indicators `is_reported/was_reported/report_cleared` · verification tier + receipts (§8.12) · trust panel).

[GATED APPROVAL] — SHARED/ORG PRODUCTION MEMORY · EXPLICIT USER ACTION ON EVERY CANDIDATE

ACCEPT → inject context + attest serve on-chain

DENY → block for this session/context (no corpus signal)

BLOCK → personal blacklist + corpus denial signal

REPORT → file on-chain report (`is_reported`); the memory keeps serving until resolved

Benchmark/test mode (outside shared-memory safety contract): evaluation mode may auto-approve candidates for throughput testing; never enabled for production shared/org recall.



Agent continues, with or without memory.

Diagram 3. Recall is queried on every prompt; shared-memory production flow uses a single mandatory human gate, with benchmark/test mode explicitly out of contract.

06 Local Architecture

MCP Plugin + Sidecars

6.1 Local Footprint and Responsibilities

Local components are the MCP server + plugin, Umbral sidecar, and wevibe-guard. The local path owns decryption, sanitization, approval, and injection.

RECALL-SIDE (LOCAL)	CONTRIBUTION-SIDE (LOCAL)
1. Harvest session signals and build the deterministic need-card.	1. Extract session learnings only when the contributor clicks <code>/sessions</code> → Extract Memories (client-side candidate generation, no submission yet).
2. Compute the query embedding locally via Ollama (<code>nommic-embed-text</code>).	2. Sanitize, encrypt, and sign submission material.
3. Send the query vector (plus context filters) to the hub retrieval endpoint.	3. Submit commitment data to the chain (organization pays submission bandwidth), then follow the moderation/finalization flow.
4. Fetch candidate ciphertext (chain is source of truth; hub serves it cached with Umbral PRE materials).	
5. Decrypt locally through the Umbral sidecar.	
6. Run wevibe-guard sanitization/policy checks.	
7. Present the human approval gate and inject only approved context.	

Table 11. Vector retrieval itself is hub-side; the hub stores and serves ciphertext, vectors, and keyword metadata, and never decrypts plaintext.

6.2 Dependencies

The MCP/plugin stack requires Ollama, a local Umbral sidecar, wevibe-guard, and connectivity to hub APIs and chain RPC.

6.3 Sync and Bootstrapping

Member setup is key/bootstrap first: join organization, receive sealed envelopes, initialize local services. Recall then runs request/response against hub retrieval; clients do not download full corpora.

6.4 Contribution Flow

DIAGRAM 4 · CONTRIBUTION FLOW

1 Developer opens the dashboard `/sessions` and selects a captured local session.

GATE – click "Extract Memories": client-side extraction with the contributor's selected model – local or session-matched cloud; returns review candidates; does NOT submit. Candidates from goal-sealed trajectories carry their receipts and provisional tier (§8.12).

3 Review/edit candidates and choose a per-memory organization destination.

GATE – click "Submit N": explicit off-machine send.

5 · Local contribution pipeline

- 1 Run wevibe-guard scan (advisory) and sanitization steps.
- 2 Generate a fresh DEK, encrypt plaintext, seal to PK_mod(e).
- 3 Contributor signs the submission hash / canonical body.
- 4 Submit COMMITMENT to the chain (hash, org_id, contributor_pubkey, expiry, size). The organization pays bandwidth for the commitment transaction.
- 5 Deliver the encrypted blob to the reviewer via a temporary off-chain path (local transfer, P2P, or org-hosted mailbox).
- 6 Return: 'WeVibe: submitted N learning(s). Pending review.'

Diagram 4. Two explicit consent points; nothing leaves the machine before Submit.

6.5 Reviewer Flow

Moderation and review run in the hosted dashboard (`wevibe-dashboard`). Reviewers and leaders process pending submissions, then finalize through the hub-to-chain path.

6.6 Plugin Architecture

AGENT	PLUGIN TYPE	STATUS
OpenCode	JS/TS editor plugin	Reference implementation

Table 12. All plugins call the same local MCP server and the same wevibe-guard binary; decryption runs through the local Umbral sidecar; retrieval/search stays in hub APIs.

6.7 Chain-Resolved Hub Endpoints (Zero-Config Transport)

The canonical chain is the organization directory. Each organization publishes ordered `hub_endpoints` (transport URLs) on-chain; `hub_serving_address` is a distinct authorization key.

Clients resolve endpoints from chain RPC at session start, verify hub response signatures against on-chain serving keys, and fail over by priority if a transport fails.

KEY • HUB TRANSPORT IS UNTRUSTED

Transport is untrusted; response authority is chain-resolved and signature-verified.

07 Content Review and Accountability

7.1 Review Flow

Contributions enter encrypted pre-commit review visible to reviewers and leaders. Pre-commit memories remain off-chain until leader commit; this is both quality control and provenance protection.

KEY • MODERATION IS ADVISORY; THE LEADER IS THE BACKSTOP

Reviewer recommendations are advisory metadata. The leader remains sole publisher: keyword curation, commit, and terminal deny are leader decisions and leader signatures.

DIAGRAM 5 • PRE-COMMIT LIFECYCLE

pending_keyword – Contributor submits; the memory enters the leader's curation queue with keywords already attached.

→

pending_chain – The leader curates keywords and verifies.

→

committed – The leader signs the multi-message batch commit.

denied – A leader denial is the only terminal removal before commit.

Advisory votes ride alongside as metadata at any point before commit.

Diagram 5. The pre-commit lifecycle. Only the leader's signature publishes; only a leader denial removes.

7.2 What Review Can and Cannot Catch

CAN CATCH	CANNOT CATCH
Prompt-injection patterns, malicious URLs, credential exfiltration, spam, duplicates, off-topic content, obvious technical errors, stale references, Unicode steganography, memories too generic for the organization's target model/stack.	Subtly incorrect technical guidance that looks plausible; semantically-encoded malicious instructions in natural-language prose; context-dependent correctness.

Table 13. What human review catches, and the semantic residue it cannot.

7.3 Reviewer Trust Boundary

Reviewers process plaintext on local endpoints; endpoint security and judgment remain part of the boundary. The leader is sole chain signer for commits, report resolutions, and dispute publications.

7.4 The Contributor-Signed Canonical Body (Verification Anchor)

Every memory's submit-time canonical body includes three fields that, together with the contributor's signature over the body, form the public-escalation verification anchor:

DEFINITION The anchor fields

- `plaintext_hash` – `sha256(salt || plaintext)`, computed by the contributor before encryption.
- `salt` – a fresh 32-byte random value generated per submission.
- `ciphertext_hash` – `sha256(ciphertext)`, where ciphertext is the AEAD output.

The contributor signs the canonical body with their own key. The canonical body, the signature, and the ciphertext all travel through moderation and land on the chain together. The leader's batch commit includes the contributor's signature; the leader cannot modify the signed fields without invalidating that signature.

This is what makes the public report tier (§7.5) trustworthy without trusting the leader. Any future reveal of plaintext + salt can be verified against the on-chain `plaintext_hash` by a direct SHA-256 check, and the on-chain ciphertext can be verified against `ciphertext_hash`. The leader is removed from the verification chain entirely; a captured leader cannot poison the anchor because they do not sign it. The contributor cannot substitute ciphertext between submit and commit (the signature binds the specific `ciphertext_hash`), and cannot later claim different plaintext at escalation (the `plaintext_hash` binds them, and SHA-256 collision resistance prevents a second matching pair).

KEY · WHY THE SIGNATURE COVERS ALL THREE FIELDS JOINTLY

A signature over `plaintext_hash` alone – without salt and without ciphertext binding – is vulnerable to contributor-plus-leader collusion: the contributor signs one hash while the leader commits ciphertext encrypting different content, and the asymmetry is undetectable. Binding all three fields in one signature closes the gap. The leader has no signing role in the anchor; the contributor has no opportunity to substitute ciphertext after signing.

7.5 The Report Model

Reports are the high-friction accountability primitive: filing with a fixed window, then public reveal only if dismissed or unaddressed.

DIAGRAM 6 · REPORT LIFECYCLE AND THE EXPOSE GATE

Stage 1 — File (no reveal)

Consumer files from the plugin. Wallet-gated: the reporter signs the report tx from their own wallet and pays gas; their public wallet is recorded on-chain.

`sets is_reported` `sets was_reported (permanent)`

- Rate-limited to one report per reporter per 24 hours (block-height scoped).
- Broadcasts directly to chain RPC; a WeVibe relay is retry-fallback only.
- The memory keeps serving during the window.

 **One-week resolution window**

Stage 2 — Resolution (org acts)

Valid → the leader deletes the ciphertext blob. A minimal on-chain transparency record is kept (reported and upheld, attributed to the contributor). Plaintext is NOT revealed.

Dismissed → the leader clears the open report with a `clear_report` tx (`report_cleared = true, is_reported = false; was_reported` stays true).

Stage 3 — Expose (gated, final) — triggered by "window elapses with no leader action" OR "leader dismisses"

The reporter's public report unlocks and reveals the memory plaintext, anchored to the contributor-signed hash (§7.4). The block explorer renders it with full provenance – contributor pubkey, leader pubkey, org ID, original commit height, and the reporter's signed reason. One escalation per report; no re-publishing.

Diagram 6. Revealing is last because it is irreversible. A captured organization cannot both refuse to act and prevent the public record.

7.6 The Expose Gate, Dispute, and Silent Acquiescence

Ordering is deliberate: file without reveal, allow one-week resolution, unlock reveal on dismissal/inaction, then allow one immutable leader counter-statement.

KEY • SILENT ACQUIESCENCE

A leader who neither upholds nor dismisses during the window implicitly accepts the claim: reveal unlocks, and the on-chain report remains visible.

7.7 No Internal Courtroom, and Silent Denial as a Cheap Signal

WeVibe's dashboard is not the courtroom. The chain is. WeVibe surfaces do not aggregate or rank report statistics. The chain is publication; public judgment happens externally via explorer URLs.

*There is no tribunal. There is only **publication of unforgeable evidence** on a chain no one controls.*

Block as low-friction negative signal. Block is silent and frictionless; it feeds Earned-Trust decay and local suppression. Reports remain high-friction and wallet-gated.

DEFINITION Earned-Trust decay

Per-memory keyword weights evolve each chain epoch: serves boost, denials decay, idle memories decay. A memory archives once every keyword weight falls below the retrieval threshold. Decay is epoch-driven.

Serve/denial settlement is relay-driven hub→chain batching; local retrieval mirrors the same arithmetic between settlements.

08 Provenance, Reputation, and the Social Graph

8.1 Reputation Is Provenance Made Visible

Reputation is a presentation layer over signed chain events: what was contributed, where it served, and under whose curation.

8.2 Serve Attribution Is a Social Signal, Not an Economic One

Serve attribution is public and on-chain, but decoupled from payout.

8.3 Identity and Attribution Model

KEY	ROLE
global_contributor_key	Public identity for authorship and contributor-profile reputation.
org_serve_key	Per-organization pseudonymous key used for retrieval/serve attribution events.

Table 14. The `org_serve_key` proves organization activity and supports deduplication without auto-linking retrieval behavior across organizations. Users can opt in to publicly link selected organization activity to a profile.

8.4 Open-Source Social Graph Client

The social graph is an open-source, forkable client over chain RPC; chain state remains source of truth.

8.5 Badge Taxonomy and Display Scope

Badge taxonomy is normative but non-economic. Canonical badge families and display semantics are specified in Appendix B.

8.6 Badge Scoping and Canonical Criteria

Badges are earned per organization and optionally summarized on contributor profiles. No global cross-organization leaderboard is published.

8.7 Signal Integrity and Anti-Gaming

Integrity controls include deduplication, per-epoch serve caps, self-serve discounting, diminishing returns, and per-entry signature verification. Human review and denial/report signals provide additional negative feedback.

8.8 Public Discovery Interface (Opt-In)

VISIBLE TO NON-MEMBERS (IF PUBLIC)	NOT VISIBLE TO NON-MEMBERS
Organization name, specialization, description, memory count, member count, age, leader identity, total serves, social-badge summary, and the two unfakeable org-health signals below.	Memory content (encrypted on-chain), member identities (privacy-preserving), review history, payout rules.

Table 16. Discovery is opt-in per organization.

KEY • TWO UNFAKEABLE ORG-HEALTH SIGNALS

- **Leader last active.** Timestamp of the latest leader-signed on-chain action.
- **Voluntary departure rate.** Share of members who left voluntarily over the trailing window.

Discovery surfaces do not aggregate report statistics.

8.9 Leader and Member Interfaces

Leader surfaces include review queue, member management, taxonomy, relay status, and report-response controls. Member surfaces include role, contribution/serve summaries, and badge progress.

8.10 Reporter's Private View

Each reporter has a private list of their own reports and published escalations.

8.11 Session Provenance: The Honest Boundary

Every provenance claim in this section rests on signatures over *submitted content*: the contributor signs the canonical body (§7.4), the leader's bonded signature publishes it, and serve history accrues to it on-chain. That chain of signatures proves **who submitted and who curated**. It does not cryptographically prove that the underlying coding session occurred as described.

⚠ WARNING · WHAT PROVENANCE DOES NOT YET PROVE

A contributor could fabricate a plausible "session" and submit memories extracted from it. Today's defenses against this are human and reputational – leader review, the contributor's reputation and wallet age shown at the gate, and the report rail – and they are real, but they are judgment, not proof. Accordingly, the word "verified" anywhere in this document means leader-curated and contributor-signed; it never implies session or model attestation.

The designed answer is a cryptographic session-attestation rail (the `x/attestation` socket, held inactive in current scope; roadmap in `ROADMAP.md` §2). Hardware-attested inference can produce signed attestation receipts that bind a session – and, folded one step further, the extraction that distills it into a memory – to an attested model and the contributor's identity. Such receipts carry only cryptographic hashes, never session text, so any client can verify them without the session being disclosed; commercial confidential-inference gateways ship receipts with exactly these properties today [30][31]. When this rail ships, it arrives as an optional, per-field-graded provenance label – never a contribution gate, and never a single "certified" badge over a claim only partly proven.

8.12 Goal-Sealed Trajectory Verification

Section 8.11 fixed the honest boundary: signatures prove who submitted and who curated, never that the underlying session occurred. This section makes part of that gap irrelevant rather than closing it. Instead of proving the session, WeVibe proves the outcome – against a target fixed before the outcome existed. A memory that turns a pre-sealed check green is real knowledge even if its transcript is theater; for the class of work that admits an executable definition of done, the session-occurrence question dissolves.

The goal seal. At session start the plugin reads the project goal asserted in `AGENTS.md`, asks the contributor to confirm an executable test predicate that defines completion – often the currently failing test – and seals both, together with a hash of the starting working tree, under the contributor's signature and a signed timestamp.

CODE 7 · GOAL SEAL STRUCTURE

```
goal_seal {
  goal_id      : unique identifier; the goal stays open while the predicate is
red
  goal_text_hash : sha256(goal statement as asserted in AGENTS.md at seal time)
  predicate_hash : sha256(test command || test file contents)
  state0_hash    : sha256(working-tree manifest at seal time)
  repo_binding   : .wevibe/org.json marker reference
  sealed_at      : signed local timestamp
  contributor_sig : Ed25519 signature over all fields above
  chain_anchor    : optional anchoring transaction hash (hook; unset in MVP)
}
```

Seals are immutable. A wrong goal is never edited; it is abandoned and a new goal sealed, and every seal still predates its own outcome – which is the entire anti-backfill property. Abandoned goals still yield negative receipts.

The trajectory chain. The plugin extends its session-signal harvesting (§5.2) by hashing working-tree states into a chained record of diffs, `state_0 → state_n`. The chain belongs to the goal, not the session: sessions attach to open goals ("Continuing goal X?") and continue the chain where the previous session ended, with each session boundary stamped with the predicate's color at that point. External changes between sessions – collaborator pushes, edits made outside the plugin – are recorded as explicit chain gaps: per-memory receipts survive a gap, but the completeness claim over the chain weakens, and the receipt says so.

Goal close. When the sealed predicate passes, the goal closes and extraction unlocks over the entire chain, not the final session alone. Necessity is judged against the finish line, not the chapter: a fix from a mid-chain session that ended red can still be the load-bearing fix. Memories submitted before their goal closes carry tier T0 and are receipt-upgraded when it does.

DIAGRAM 7 · ONE GOAL, MANY SESSIONS

Goal sealed – predicate + state_0, signed, timestamped.

S1 Config change

RED

S2 Timeout raised

RED

↳ Negative receipt: present at a red state, absent from the final green state (dnd).

S3 Timeout reverted; pool recycling added

RED

↳ Ablation: revert pool recycling from final state → RED ⇒ load-bearing (T2).

S4 Retry double-release fixed

GREEN · goal closes

Extraction unlocks over the whole chain.

Diagram 7. The seal belongs to the goal, not the session. Receipts are judged against the finish line: a red chapter can still contain the decisive fix.

Receipts. Each extracted memory cites the specific diff in the chain where its insight was earned. Three receipt types attach to that citation:

RECEIPT	WHAT IT ATTESTS	STRENGTH
Predicate receipt	The sealed test passed at state_n : exit status, environment fingerprint, and chain reference, signed.	A fact at goal close: the outcome met a definition of done fixed before the outcome existed.
Ablation receipt	Reverting the memory's cited diff from the final state and rerunning the sealed predicate turns it red.	Necessity: the cited fix was load-bearing for the outcome. Best-effort — revert conflicts and flaky predicates void it; it is a graded bonus, never a rung every memory reaches.
Negative receipt	The cited change was present at state_x , the predicate ran red there, and the change is absent from the final green state.	Deliberately weak: proves "not sufficient as tried, then" — never "wrong." A masked or incomplete idea earns the same receipt as a bad one.

Table 21. Receipts are contributor-machine artifacts: checkable claims, not proofs. See the boundary callout below.

Leave-one-out ablation has a known blind spot: if either of two surviving changes alone keeps the predicate green, reverting one leaves it green and neither earns the badge, even though together they mattered. Full subset testing is combinatorial and deliberately out of scope.

Tier grading. Receipts compose into a per-memory verification tier:

TIER	REQUIRES	WHAT IT SAYS AT THE GATE
T0	Contributor signature over the canonical body (§7.4).	Today's baseline: a signed, leader-curated claim.
T1	Goal seal + trajectory chain + predicate receipt.	The outcome met a pre-sealed definition of done.
T2	T1 + ablation receipt.	The cited fix was shown necessary for that outcome.
T3	T2 + attested-run receipt through the <code>x/attestation</code> socket (§8.11).	The receipts themselves were produced by an attested runtime, not merely claimed. Roadmap-scoped (<code>ROADMAP.md</code> §2).
T4	k independent contributors, k independent seals, matching context fingerprint, convergent fix [36].	Stranger convergence: the certification is the agreement itself. A sybil must actually solve the task k times against k pre-fixed predicates. Requires corpus traffic; inert at cold start.

Table 22. Tiers are per-field, per-memory labels — rendered in review and at the plugin gate — and are never a contribution gate. Exploration, design judgment, and much negative knowledge have no executable predicate and contribute at T0 indefinitely, by design. Tiers do not enter retrieval scoring.

Surfaces. The contributor sees one banner at seal time ("Goal locked."), one question when resuming ("Continuing goal X?"), one stamp at session end, and one optional action at extraction ("Prove this memory mattered" — runs the ablation check, costs local compute, skippable). The reviewer queue gains a per-memory receipt strip: goal sealed before work ✓ / predicate passed ✓ / ablation red ✓ — or blank. The consumer gate gains the single Verification row specified in Figure 2. The four-button gate (Table 3) is unchanged, and nothing is ever blocked for lacking a receipt.

KEY Why fabrication buys nothing

A target fixed before the answer existed cannot be retrofitted to whatever was produced. And to forge a green predicate receipt, a contributor must make the pre-sealed predicate actually pass — which is the work itself. Verification cost collapses onto honest cost: proof-of-work for knowledge.

⚠ WARNING • THE HONEST BOUNDARY (CHECKABLE, NOT UNFORGEABLE)

T1-T2 receipts are produced on hardware the contributor controls and can be fabricated. Pre-commitment changes what a fabrication becomes: a forged receipt against a sealed predicate is a provable lie – reveal the sealed inputs, replay the predicate, watch it fail – anchored to the contributor's signature and handled through the same report rail as content disputes (§7.5). Replay requires the contributor's private states, so it bites only through the dispute path: a demanded reveal that is refused reads as silent acquiescence (§7.6), not as automatic detection. Two limits are permanent. A trivially weak sealed predicate yields technically true, practically empty receipts – a tier badge says "met its own pre-set bar," never "the bar was high"; bar quality remains with leader curation and serve/Block signals (§7.7). And no tier below T3 attests how the receipts were produced.

09 Storage Architecture

9.1 On-Chain Encrypted Memory Storage

Approved memories are encrypted chain state: ciphertext, wrapped DEK, plaintext metadata, and batch Merkle leaves. Validators replicate all memory state.

KEY • SIZE ECONOMICS

Typical encrypted memory size is 500 bytes–2 KB. At 10,000 memories: ~10–20 MB chain state; at 100,000: ~100–200 MB.

9.2 Keyword Index (On-Chain Metadata)

Per-memory keyword weights are on-chain metadata. Taxonomy management is hub-side but anchored by on-chain `vocab_hash` and `embedding_model_id`.

9.3 Semantic Vector Index (Hub Qdrant)

Embeddings are hub-side derived data, rebuildable from chain state plus organization keys. Qdrant holds vectors + keyword metadata, not plaintext memory bodies; the confidentiality boundary and semantic-shadow concession are defined once in §4.5.

9.4 Memory Metadata

FIELD	MEANING
<code>contributor_pubkey</code>	On-chain identity of the contributor.
<code>model_origin</code>	Contributing model.
<code>stack_tags</code>	Freeform technology tags.
<code>version</code>	Nullable version string.
<code>source</code>	<code>session</code> <code>doc_import</code> <code>authored</code> .
<code>approved</code>	Boolean moderation state.
<code>is_reported</code>	An open report currently stands against this memory.
<code>was_reported</code>	Reported at least once; permanent historical flag.
<code>report_cleared</code>	The leader dismissed the open report via <code>clear_report</code> .
<code>quarantined</code>	Flag for memories with repeated upheld rejections; retrieval-policy exclusion.
<code>deprecated</code>	Curator has marked stale.
<code>verification_tier</code>	T0–T4 receipt tier at commit time; upgradeable on goal close (§8.12).
<code>receipt_refs</code>	Hashes of the goal seal, predicate receipt, and (if present) ablation receipt. Nullable.
<code>mc_version</code>	Memory Contract (MC-1) schema version.

Table 17. The per-memory metadata record.

10 Security Analysis

10.1 Sybil Resistance

Free identity creation is not prevented; harm is gated at organization membership. Membership and publication are leader-gated, and contributor reputation is resettable on removal.

10.2 Memory Poisoning

Canonical defenses and limits are specified in §4.7 (sanitization pipeline, Table 10) and §7.2 (review coverage limits). Residual risk remains semantic payloads and plausible-but-wrong guidance.

10.3 Leader Key Compromise

`K_master` compromise exposes epoch-derived content. Mitigations are recovery phrase handling, encrypted vaulting, threshold recovery, and on-chain serving-key verification.

10.4 Chain State Observability

On-chain metadata is public; plaintext and local private state are not.

10.5 Network-Level Anti-DDoS

Alpha anti-DDoS is enforced by `x/bandwidth` per-organization per-epoch caps. Mainnet economic anti-spam is roadmap-scoped.

10.6 Content Suitability Policy

SUITABLE	UNSUITABLE
Coding patterns and anti-patterns, architecture lessons, debugging notes, dependency guidance, tool usage, process workflows, version-specific gotchas, negative knowledge.	Credentials/secrets, customer PII, regulated data, legal/HR records, high-sensitivity security-incident details.

Table 18. What belongs in a WeVibe memory, and what does not.

10.7 Organization Capture and Public Escalation

A single actor wearing every hat – leader, every member with `can_moderate`, every contributing member – can fully capture an organization's internal governance. Inside the organization, every approval, every report dismissal, and every chain commit can be coordinated. Internal accountability primitives (advisory votes, dispute counts, internal review queues) provide no protection against this: the captured operator simply approves their own malicious memories and dismisses every report against them.

The system's security model is therefore not "prevent capture through internal governance." It is:

*Make capture **economically unsustainable** through transparent on-chain accountability, frictionless exit for members, and a public escalation primitive a captured organization cannot suppress.*

DEFINITION Four load-bearing properties

- **The chain is the unforgeable audit log.** Every consequential action – memory commit, denial settlement, report resolution, dispute publication, member departure – is a signed on-chain transaction. Neither the captured organization, nor WeVibe, nor any platform operator can edit or suppress it after the fact.
- **Consumers hold an escalation path the organization cannot close.** A dismissed (`clear_report`) or unaddressed on-chain report unlocks a reporter-signed public escalation – wallet-gated, gas-paid, revealing plaintext only after the one-week window elapses or the leader dismisses, anchored to the contributor-signed hash the leader cannot poison (§7.4) – and once published it cannot be edited or deleted. The reporter broadcasts the filing and expose directly to chain RPC, with the WeVibe relay as retry fallback only.
- **Exit is unfakeable.** Members leaving voluntarily is a first-class on-chain event. Sybils can be invited and can file frivolous reports, but they cannot fake people walking away. The voluntary-departure-rate signal on discovery (§8.8) lets prospective joiners read the most honest possible signal.
- **Hub compromise is a per-organization degradation event, not a network takeover.** Per-memory Umbral crypto and consumer-side wevibe-guard still gate plaintext and injection, and hub responses must verify against on-chain serving keys. A compromised endpoint can at worst degrade or poison recall for that one organization; it cannot mint identities, steal contributor keys, or affect other organizations. The endpoint can be rotated on-chain by leader signature.

The worst case in this class is degraded or poisoned recall quality for one organization, typically surfaced by guard/crypto checks and reversible by on-chain endpoint rotation.

The leader bears sole signature. Co-attestation of additional reviewer public keys on leader-signed chain transactions is explicitly excluded (§7.3). A leader's chain commit binds the leader's wallet only. This concentrates responsibility on the actor who actually signs and prevents implicating advisory reviewers in chain-level decisions they did not authorize.

Why no platform tribunal. WeVibe deliberately does not adjudicate published reports. Any in-app judging body is a capture vector – whoever controls the tribunal controls the verdict. The chain publishes the evidence, the block explorer renders it, the reporter shares the URL, and the public judges on its own merits (§7.7).

⚠ WARNING • RESIDUAL: CONTRIBUTOR-LEADER COLLUSION

If the contributor who submitted a memory and the leader who committed it are both adversaries, the contributor can sign a false plaintext hash, the leader can commit it, and a public report's plaintext reveal will not match the on-chain hash – making a true report look invalid. The on-chain ciphertext + capsule remain a final backstop: any future key disclosure (epoch rotation, organization closure, legal discovery) lets independent parties decrypt and verify after the fact (§7.4). This residual cannot be eliminated at submit time – when the content creator and the content approver are the same adversary, there is no honest party in the verification chain to sign over.

10.8 Receipt Integrity and Predicate Gaming

Verification receipts move risk in one direction: they convert unverifiable narrative into checkable claims. Canonical semantics, tiers, and limits are specified in §8.12. Residual risks are three: fabricated T1–T2 receipts, detectable only through dispute-path reveal-and-replay; trivially weak sealed predicates, bounded by "met its own bar" label semantics and leader curation rather than cryptography; and leave-one-out ablation blindness to jointly redundant fixes. None of these widens the injection surface: tiers are labels rendered at the human gate, never bypasses of it, and a memory with no receipt is exactly as admissible as it is today.

11 Decentralized Architecture

11.1 Chain Architecture

WeVibe runs as a sovereign L1 appchain on Cosmos SDK + CometBFT with deterministic finality and safety-over-liveness behavior. Organization state includes chain-resolved `hub_endpoints`; serving authority is separate (`hub_serving_address`).

11.2 The Four Roles in Detail

- **Developer.** Onboards by passkey, contributes explicitly, recalls through the mandatory gate.
- **Organization leader.** Signs org registration from their own wallet and is sole publishing authority for commits.
- **Validator.** Runs consensus and stores encrypted chain state.
- **WeVibe (protocol).** Open-source software; no single operator is a required trust anchor.

11.3 On-Chain Modules

MODULE	RESPONSIBILITY
x/org	Slot registry, membership, chain-resolved transport/auth fields (hub_endpoints , hub_serving_address), feegrant, dormancy/abandonment detection.
x/memory	Pending commitments, approved encrypted blobs, epoch Merkle roots, contributor-signed anchors, report/quarantine flags.
x/serve	Batched serve/denial recording, entry-signature verification, deduplication, self-serve handling, social attribution aggregation.
x/identity	Paskey-derived contributor identity and key registry.
x/reputation	Cross-organization contributor aggregates (serve count, breadth, domain tags, rep score).
x/emissions	Validator rewards, contributor emissions, schedule controls, claim path.
x/bandwidth	Per-organization per-epoch flat rate-limit caps in testnet/alpha anti-DDoS mode.
x/attestation	Session-attestation storage socket (inactive in current scope; §8.11, ROADMAP.md §2). Reserved anchor point for optional goal-seal and verification-receipt commitments (§8.12; hook only in MVP).

Table 20. Custom modules. Standard SDK modules also in use: [x/staking](#), [x/auth](#), [x/bank](#), [x/gov](#), [x/slashing](#), [x/distribution](#).

11.4 Node Lifecycle ([wevibed](#))

[wevibed](#) follows standard Cosmos bootstrap patterns.

12 References

- 1 Song, D., Wagner, D., Perrig, A. (2000). *Practical Techniques for Searches on Encrypted Data*. IEEE S&P.
- 2 Curtmola, R., et al. (2006). *Searchable Symmetric Encryption*. ACM CCS.
- 3 Cormack, G. V., et al. (2009). *Reciprocal Rank Fusion*. SIGIR '09.
- 4 Reimers, N. & Gurevych, I. (2019). *Sentence-BERT*. EMNLP 2019.
- 5 Kusupati, A., et al. (2022). *Matryoshka Representation Learning*. NeurIPS 2022.
- 6 Greshake, K., et al. (2023). *Indirect Prompt Injection*. arXiv:2302.12173.
- 7 OWASP (2025). *Top 10 for LLM Applications*.
- 8 Sha, F., et al. (2025). *Lessons from Defending Gemini Against Indirect Prompt Injections*. Google DeepMind.
- 9 Zou, A., et al. (2024). *PoisonedRAG*. arXiv:2402.07867.

- 10 Google DeepMind (2026). *AI Agent Traps*.
- 11 *MLS Architecture* (RFC 9420).
- 12 Lambda Class. *CommitLLM: Cryptographic commit-and-audit for LLM inference*. github.com/lambdaclass/CommitLLM.
- 13 Morris, J., et al. (2023). *Text Embeddings Reveal (Almost) As Much As Text*. EMNLP 2023.
- 14 Huang, Y., et al. (2024). *Transferable Embedding Inversion Attacks*. ACL 2024.
- 15 Núñez, D., et al. (2018). *Umbral: A Threshold Proxy Re-Encryption Scheme*. NuCypher / NICS Lab, Universidad de Málaga.
- 16 OpenAI. *ChatGPT Memory – Memory FAQ / "Dreaming."* help.openai.com/en/articles/8590148-memory-faq (accessed July 2026).
- 17 Anthropic. *Claude Memory + Projects / Memory Tool*. claude.com/blog/memory (accessed July 2026).
- 18 Google. *Gemini Personal Intelligence / memory*. support.google.com/gemini/answer/16598469 (accessed July 2026).
- 19 GitHub. *About GitHub Copilot Memory*. docs.github.com/en/copilot/concepts/agents/copilot-memory (accessed July 2026).
- 20 Bhartendu-Kumar et al. *Rules & Memory-Bank pattern (Cursor / Windsurf / Cline / Roo)*. github.com/Bhartendu-Kumar/rules_template (accessed July 2026).
- 21 Mem0. *Mem0 – memory layer for AI agents*. github.com/mem0ai/mem0 (accessed July 2026).
- 22 Letta AI. *Letta (formerly MemGPT)*. github.com/letta-ai/letta (accessed July 2026).
- 23 Zep AI. *Zep / Graphiti – temporal knowledge-graph memory*. github.com/getzep/graphiti; arXiv:2501.13956 (accessed July 2026).
- 24 LangChain. *LangMem / LangGraph long-term memory*. langchain-ai.github.io/langmem (accessed July 2026).
- 25 *RAG-over-a-wiki (Notion/Confluence + vector DB)*. Widely-used integration pattern; no single-vendor specification (accessed July 2026).
- 26 Recall Network. *Recall – decentralized intelligence layer*. recall.network (accessed July 2026).
- 27 Ceramic Network. *Ceramic / ComposeDB*. github.com/ceramicstudio/ceramic-ai (accessed July 2026).
- 28 Arweave / AO. *Permanent storage protocol + compute layer*. arweave.com (accessed July 2026).
- 29 GaiaNet. *GaiaNet – GenAI agent network (litepaper)*. docs.gaianet.ai (accessed July 2026).
- 30 TLSNotary (Privacy Stewards of Ethereum). *TLSNotary – data provenance without compromising privacy*. tlsnotary.org/docs (accessed July 2026).
- 31 RedPill / Phala. *Confidential AI inference: attestation reports and signed receipts*. docs.redpill.ai; docs.phala.com (accessed July 2026).
- 32 Apple Security Engineering and Architecture. *Private Cloud Compute: A new frontier for AI privacy in the cloud*. security.apple.com/blog/private-cloud-compute/ (June 2024; accessed July 2026).
- 33 Google. *Private AI Compute*. blog.google/innovation-and-ai/products/google-private-ai-compute/ (accessed July 2026).

- 34 *Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents.* arXiv:2606.15017 (accessed July 2026).
-
- 35 Wang, Z., et al. (2025). *Agent Workflow Memory*. ICML 2025; arXiv:2409.07429.
-
- 36 *WISE-Flow: Workflow-Induced Structured Experience for Self-Evolving Conversational Service Agents.* arXiv:2601.08158 (accessed July 2026).
-

A Appendix A — Cryptographic Procedures

Each organization has a master key `K_master`. Per-epoch keys derive from it via HKDF. The leader derives these keys locally.

CODE 1 · KEY DERIVATION

```
K_enc(e)    = HKDF-SHA256(K_master, info="wevibe-enc-" || epoch_be_bytes)
K_audit(e)  = HKDF-SHA256(K_master, info="wevibe-audit-" || epoch_be_bytes)
epoch_sk(e) = HKDF-SHA256(K_master, info="wevibe-umbral-epoch-" ||
epoch_decimal_ascii)[:32] → Umbral secret key
```

- `K_enc(e)` wraps per-memory DEKs for approved memories in epoch e .
- `K_audit(e)` is reserved for audit and receipt verification.
- `epoch_sk(e)` is derived on the leader machine and never transmitted; the hub receives only `umbral_pk` and finished kfrags.

Each memory is encrypted with a fresh DEK.

CODE 2 · MEMORY ENCRYPTION

```
DEK          = random(32)
ciphertext    = AES-256-GCM(DEK, nonce, plaintext_memory)
wrapped_dek_mod = seal_to_pubkey(DEK, PK_mod(e))
```

On approval, the reviewer re-wraps DEK under the epoch encryption key.

CODE 3 · APPROVAL RE-WRAP

```
wrapped_dek_enc = AES-256-GCM(K_enc(e), nonce, DEK)
```

The approved bundle (`ciphertext` , `wrapped_dek_enc` , metadata) is then published on-chain.

CODE 4 · SEAL_TO_PUBKEY

```
seal_to_pubkey:
  1. Generate an ephemeral X25519 keypair.
  2. ECDH between the ephemeral private key and the recipient's X25519 public key.
  3. Derive a symmetric key via HKDF (info: "wevibe-envelope-v1").
  4. Encrypt with AES-256-GCM.
  5. Output: ephemeral_pubkey (32 bytes) || nonce (12 bytes) || ciphertext+tag
```

Custody model invariant. Each organization's `K_master` is independent random entropy.

Recovery. BIP39 24-word recovery phrase; Shamir 2-of-3 threshold recovery; encrypted leader vault (`~/.wevibe/vault.enc`) with Argon2id (t=3, m=64 MB, p=4).

B Appendix B — Badge & Reputation Taxonomy

FAMILY	BASIS
Serve-milestone badges	Thresholds on how often a contributor’s memories are served.
Rarity-tier badges	Per-memory keyword supply/demand tiers, computed read-time by the social-graph client.
Contribution-volume badges	Thresholds on approved-memory contribution volume.

Table 15. Scoping is per-organization, with profile breakdowns instead of a single global ladder. Badge status is strictly non-economic: no reward and no payout coupling.

Organization-level badge state is canonical. Contributors may optionally summarize across organizations, but WeVibe does not publish a global leaderboard.

c Appendix C — The Document Set

WeVibe is a family of interoperable services. This general architecture document captures the shared threat model, encryption design, and contributor experience. Deeper, implementation-specific material lives alongside the source for each product: the consensus-layer economics and keeper architecture; the client stack (SDK, MCP server, guard, protocol assets) and plugin UX; and the operational surfaces (hub, dashboard, infrastructure) with their deployment models. Each product set also carries its own protocol-detail and topology documents. Use this document for cross-cutting concerns and the per-product sets for implementation specifics. Forward-looking capabilities and the token-economic model are specified separately in [ROADMAP.md](#).

This document is an architecture specification. Nothing in this document constitutes an offer or solicitation of securities.